

CSE 221: Algorithms

Chapter 3 — Searching Algorithms

Prepared by Ariyan Hossain [AYO]

Searching means finding whether an item exists in a collection (e.g., an array) and, if it exists, where it occurs. This chapter covers three classic searching approaches: **linear search**, **binary search**, and **ternary search**.

1 Linear Search

Linear search is the most basic form of searching. Given an item to search for from a “bucket” of items, we check elements one by one. If we find the item, then *bingo!* Otherwise, we reach the end of the bucket.

Example

Given $A = [1, 2, 3, 4, 5]$ and the item we are looking for is 5, we scan left to right until we see 5.

1.1 Pseudocode (Linear Search)

Linear Search (iterative)

```
linearSearch(A[], item){
    int i = 0;
    while (i < A.length){
        if (A[i] == item){
            return true;
        }
        i++;
    }
    return false;
}
```

Follow-up Question

Q) What is the time complexity of linear search?

Ans: $\mathcal{O}(n)$

2 Binary Search

Binary search follows the divide-and-conquer idea. Unlike linear search, it does *not* scan the entire array. Given a sorted array and an item k to search for, we:

- compute the midpoint, mid ,
- if $A[\text{mid}] = k$, we return **TRUE**,
- otherwise, we choose one of the two segments (left / right) based on whether k is smaller or larger than $A[\text{mid}]$.

2.1 Simulation (Binary Search)

Let $A = [1, 2, 3, 4, 5]$ and $item = 5$. We use indices l (left), r (right), and m (mid):

Array state	Workings
$[1_l, 2, 3_m, 4, 5_r]$	$l = 0, r = 4$ $m = \lfloor (l + r)/2 \rfloor = 2$ $A[m] = 3 < 5$; set $l = m + 1 = 3$
$[1, 2, 3, 4_{l=m}, 5_r]$	$l = 3, r = 4$ $m = \lfloor (l + r)/2 \rfloor = 3$ $A[m] = 4 < 5$; set $l = m + 1 = 4$
$[1, 2, 3, 4, 5_{l=r=m}]$	$l = 4, r = 4$ $m = \lfloor (l + r)/2 \rfloor = 4$ $A[m] = 5 = item(5)$; TRUE

Binary search simulation for finding 5 in $[1, 2, 3, 4, 5]$.

2.2 Pseudocode (Binary Search)

Binary Search (iterative)

```

binarySearch(A[], l, r, item){
    while (l <= r){
        mid = (l + r) / 2;

        if (A[mid] == item){
            return true;
        } else {
            if (item < A[mid]){
                r = mid - 1;
            } else {
                l = mid + 1;
            }
        }
    }
    return false;
}

```

Follow-up Questions

Q) What is the recurrence equation?

Ans: It depends on the number of subproblems, each subproblem size, and the work done at each step. In each step, there is 1 subproblem where size gets divided by 2 ($n/2$) and work done is 1 at each step.

$$T(n) = T(n/2) + 1$$

Q) What is the time complexity of binary search?

Ans: $\mathcal{O}(\log_2 n)$

3 Ternary Search

Ternary search is also divide-and-conquer. Like binary search, it works on a sorted array and repeatedly narrows the search range. Given a sorted array and an item k to search for, we:

- compute two midpoints, `mid1` and `mid2`,
- if $A[\text{mid1}] = k$ or $A[\text{mid2}] = k$, we return `TRUE`,
- otherwise, we choose one of the three segments (left / middle / right) based on whether k is smaller than $A[\text{mid1}]$ or larger than $A[\text{mid2}]$.

3.1 Simulation (Ternary Search)

Let $A = [1, 2, 3, 4, 5, 6, 7]$ and `item` = 7. We use indices l (left), r (right), m_1 (mid1), and m_2 (mid2):

Array state	Workings
$[\underline{1}_l, 2, \underline{3}_{m_1}, 4, \underline{5}_{m_2}, 6, \underline{7}_r]$	$l = 0, r = 6$ $m_1 = l + \lfloor (r - l)/3 \rfloor = 2, m_2 = r - \lfloor (r - l)/3 \rfloor = 4$ $A[m_2] = 5 < 7$; set $l = m_2 + 1 = 5$
$[1, 2, 3, 4, 5, \underline{6}_{l=m_1}, \underline{7}_{r=m_2}]$	$l = 5, r = 6$ $m_1 = l + \lfloor (r - l)/3 \rfloor = 3, m_2 = r - \lfloor (r - l)/3 \rfloor = 6$ $A[m_2] = 7 = \text{item}(7)$; TRUE

Ternary search simulation for finding 7 in $[1, 2, 3, 4, 5, 6, 7]$.

3.2 Pseudocode (Ternary Search)

Ternary Search (iterative)

```
ternarySearch(A[], l, r, item){
    while (l <= r){
        mid1 = l + (r - l) / 3;
        mid2 = r - (r - l) / 3;

        if (A[mid1] == item || A[mid2] == item){
            return true;
        } else {
            if (item < A[mid1]){
                r = mid1 - 1;
            } else if (item > A[mid2]){
                l = mid2 + 1;
            } else {
                l = mid1 + 1;
                r = mid2 - 1;
            }
        }
    }
    return false;
}
```

Follow-up Questions

Q) What is the recurrence equation?

Ans: It depends on the number of subproblems, each subproblem size, and the work done at each step. In each step, there is 1 subproblem where size gets divided by 3 ($n/3$) and work done is 1 at each step.

$$T(n) = T(n/3) + 1.$$

Q) What is the time complexity of ternary search?

Ans: $\mathcal{O}(\log_3 n)$

Q) Which is faster: binary search or ternary search?

Ans: Binary search is faster in practice. Even though $\log_2 n$ is larger than $\log_3 n$, ternary search typically needs **two** comparisons per iteration (left / middle / right), while binary search needs **one** (left / right). So the rough work is:

$$\text{Binary: } 1 \cdot \log_2 n \quad \text{vs} \quad \text{Ternary: } 2 \cdot \log_3 n.$$